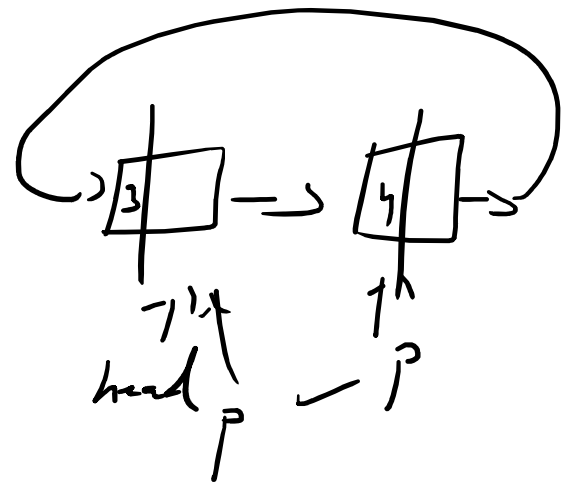
23-03-2026 - Use Linked Lists to solve problems in diverse applications.

23 March 2026 09:19

→ **Circular List** :

```
display (Node *p) {
    while (p != head) {
        print (p->data);
        p = p->next;
    }
}
```



Fatal Flaw: No while loop allowed

→ do while loop : Works with circular LL

```
display ( Node *p) {
    do {
        print (p->data);
        p=p->next
    } while (p != head)
```

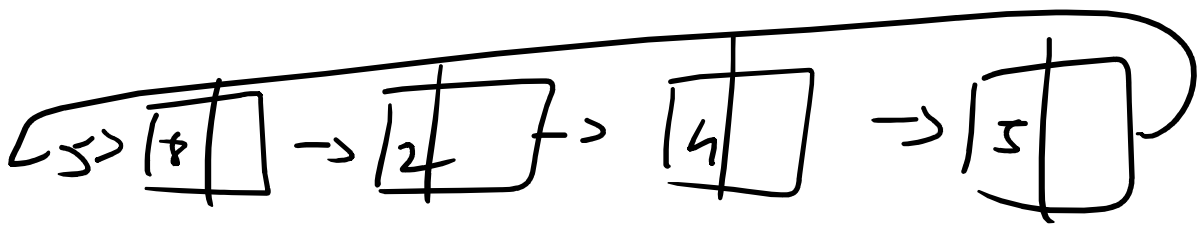
} while (p != head)

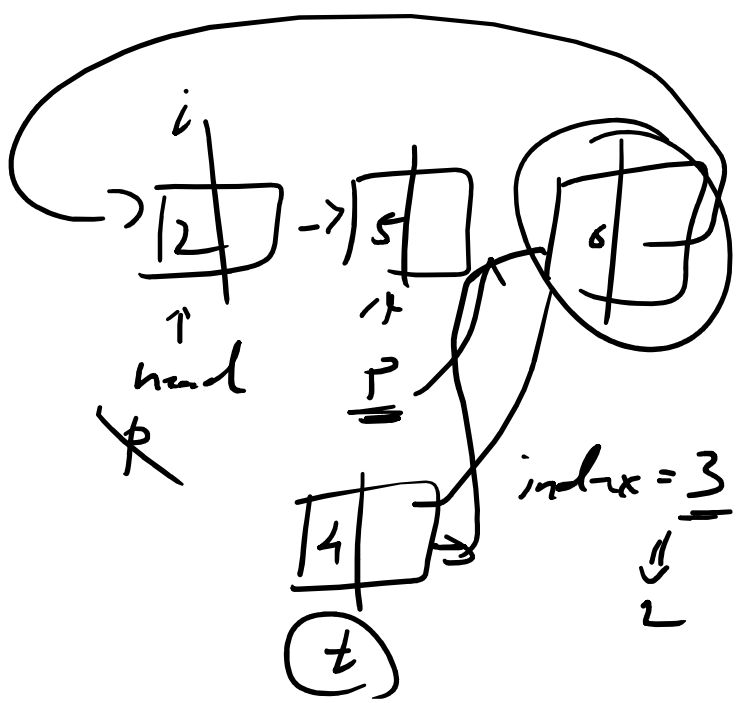
}

=> Insertion:

-> Insert before head

-> Insert at given position





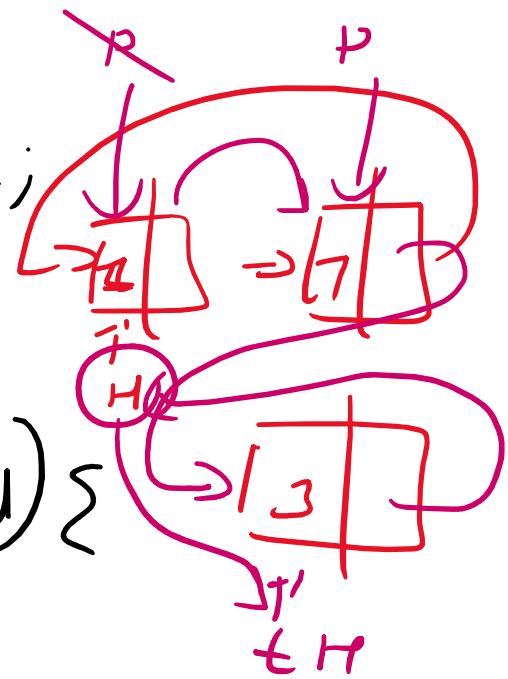

```
Node *t;
Node *p = head;
for (i=0; i<index-1; i++)
    p = p->next

t = new Node
t->data = x
t->next = p->next
p->next = t;
```

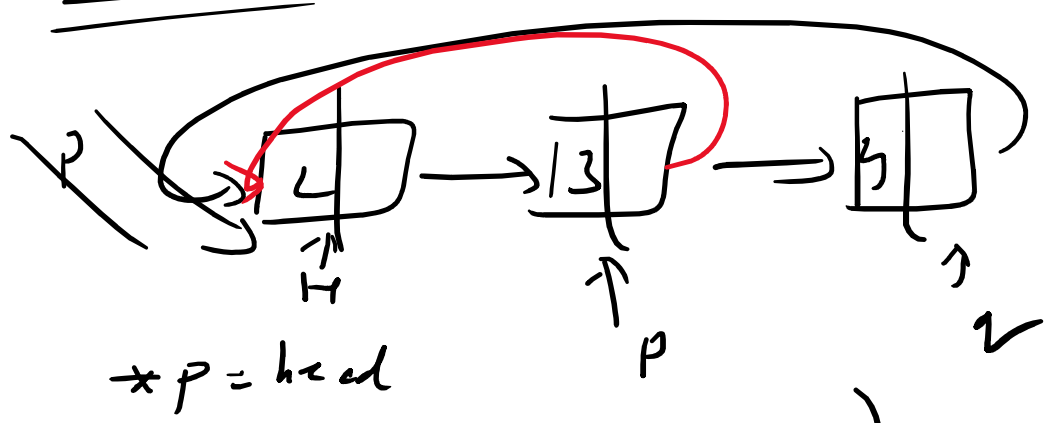
=> Inserting at head:

⇒ Inserting at head:

```
Node * p = head;
Node * t = new Node;
t -> data = x
t -> next = head;
while ( p -> next != head) {
        p = p -> next;
}
p -> next = t
head = t;
```

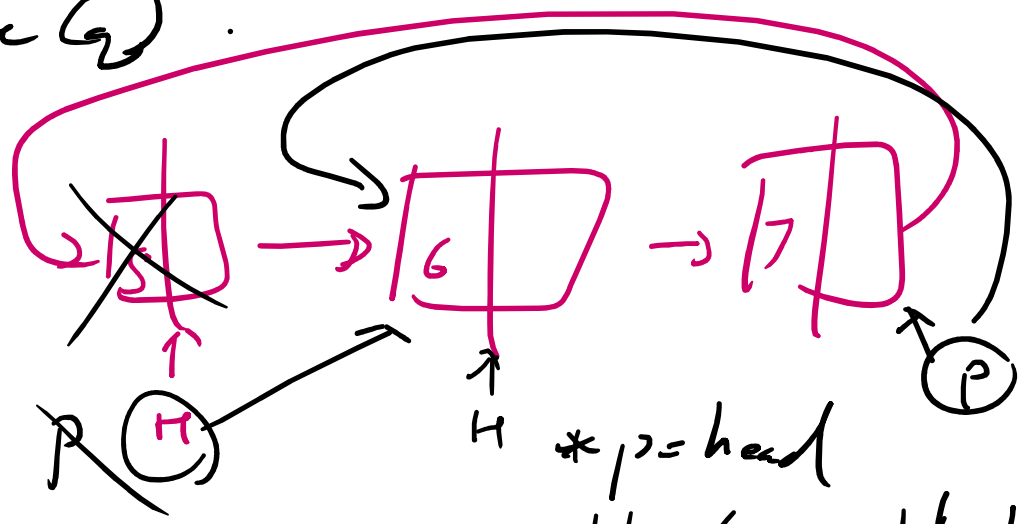


⇒ Deletion:



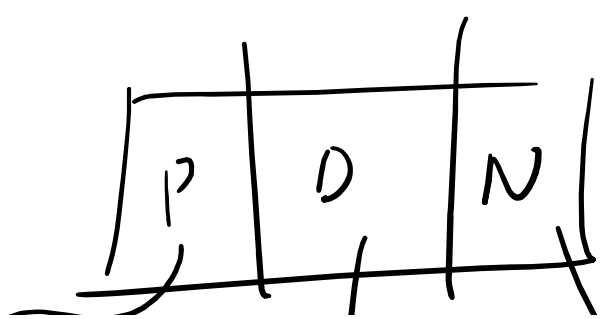

```
* p = head
for (i=0; i<index-1 ; i++)
        p = p -> next
q = p -> next
p -> next
```

```
q = p->next
p->next = q->next
x = q->data
free(q)
```



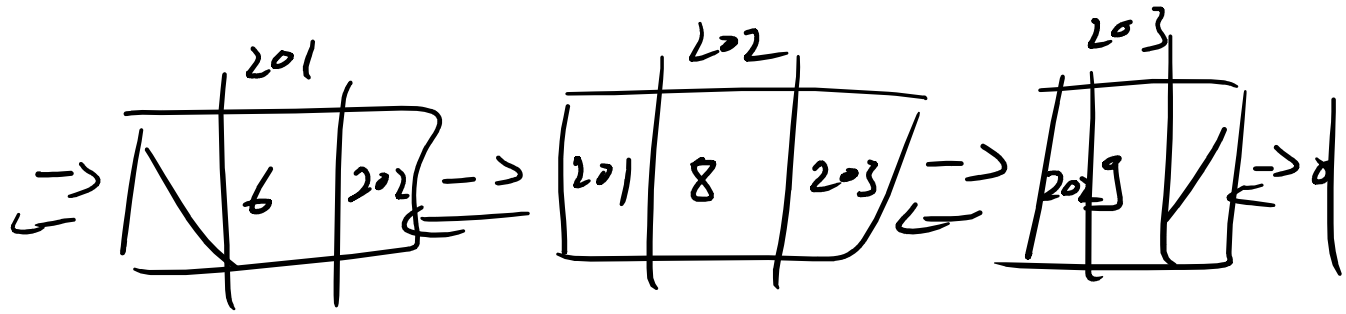

```
*p = head
while (p->next != head)
    p = p->next
p->next = head->next
x = head->data
free(head)
head = p->next;
```

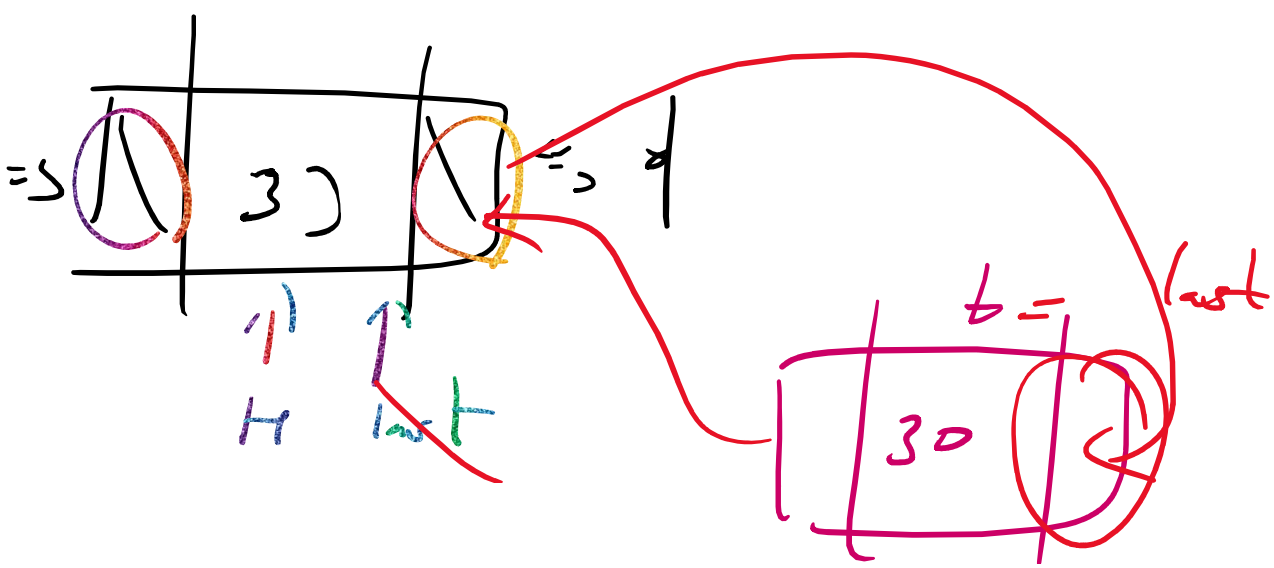
⇒ <u>Doubly Linked List:</u>



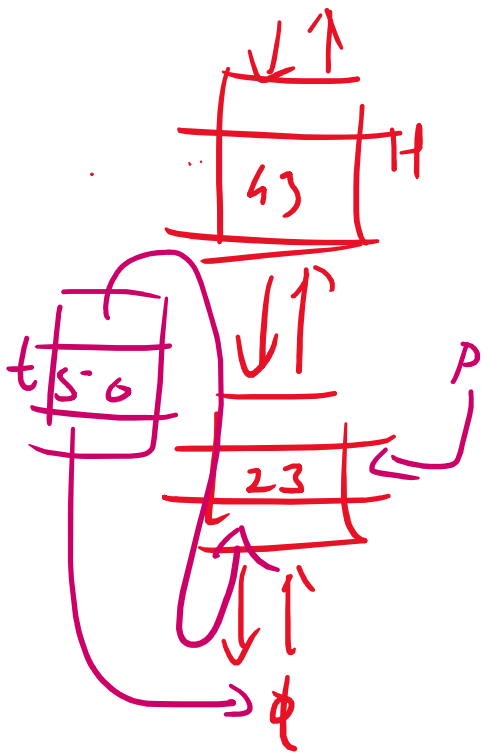
```
struct Node {
    struct Node *prev;
    int data;
    struct Node * next;

}
```





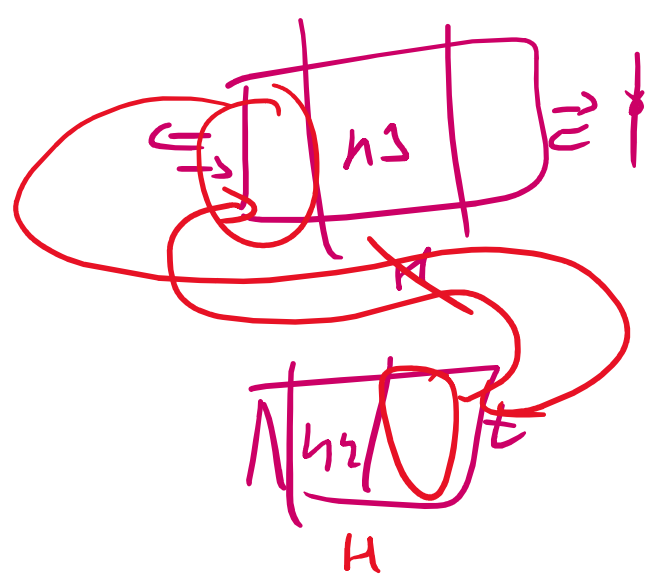
⇒ Insertion :




```
* t = new Node;
t -> data = x;
for (i=0; i<index-1; i++)
        p = p -> next
t -> next = p -> next;
t -> prev = p;
if (p -> next)
    p -> next -> prev = t
p -> next = t
```

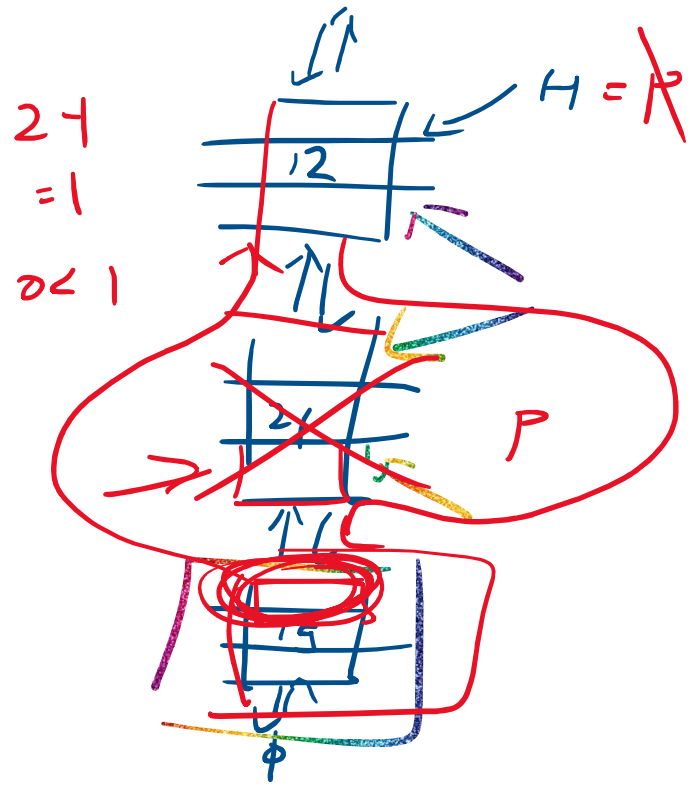
⇒ `* t = new Node`

```
t -> data = x
t -> prev = NULL;
t -> next = head
head -> prev = t
head = t
```



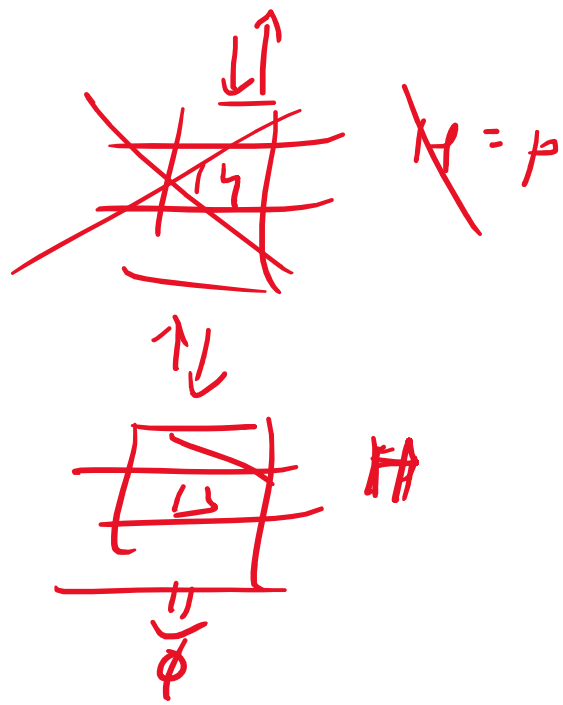
=> Deletion in Doubly LL :

```
*p = head;
for (i=0; i<index-1; i++)
    p = p->next
p->prev->next = p->next
if (p->next)
    p->next->prev = p->prev
x = p->data
free(p);
```



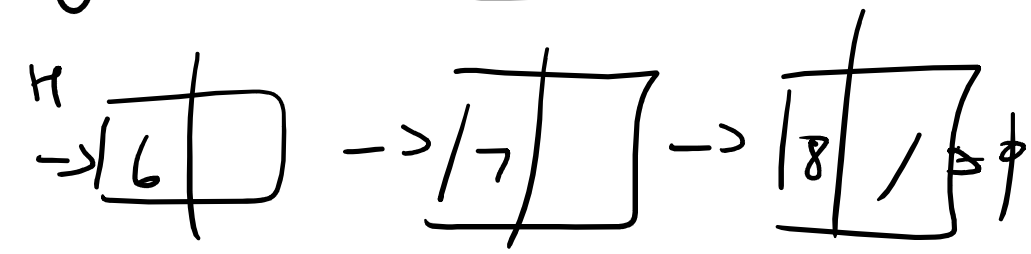
=> Deleting First Node :

```
*p = head;
head = head->next;
x = p->data;
free(p)
if (head)
    head->prev = NULL;
```



=> Reversing a linked list :

⇒ Reversing a linked list:



1) Reversing Elements

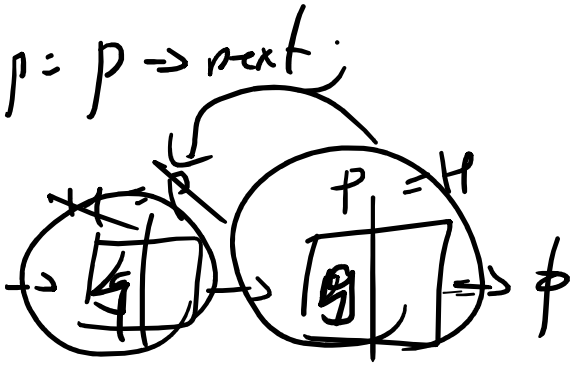
2) Reversing links

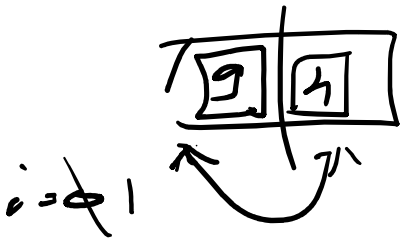
⇒ Reversing Elements:

```
while(p != NULL)
  p->data = A[i--];
  p = p->next;
```

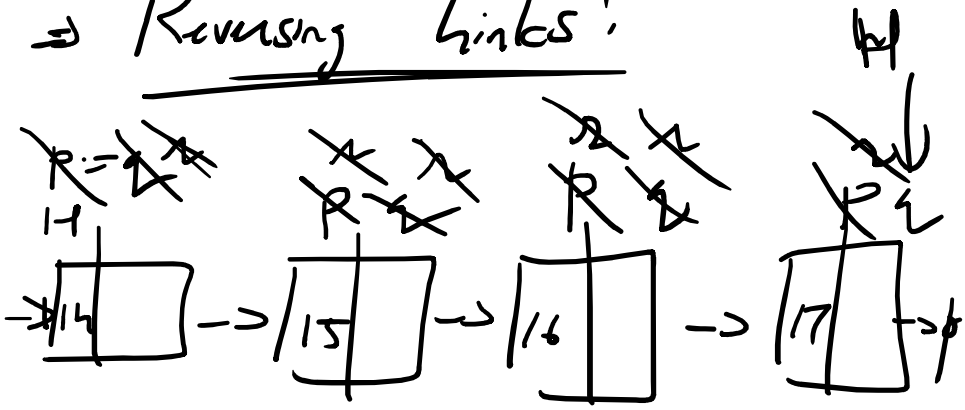
Phase 1

```
*p = head; i = 0
while (p != NULL) {
  A[i] = p->data
  p = p->next
  i++;
}
p = head; i--;
```





⇒ Reversing Links:




```
*p = head;
q = NULL;
r = NULL;
while (p != NULL){
    r = q
    q = p
    p = p->next
    q->next = r
}
head = q;
```

r = p